

ANÁLISIS DEL RETO

Juan Sebastian rojas Marin, 202610140, js.rojasm123@uniandes.edu.co

Katy Terán Mercado, k.teranm@uniandes.edu.co, 202614247

Juan José Rincón Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento 4

Descripción

```
320 def req_4(catalog, country, product):
321     """
322     start_time = get_time()
323     orders = catalog["orders"]
324     count = 0
325     prom_box = 0
326     prom_disc = 0
327     prom_mark = 0
328     prom_box_ship = 0
329     max1 = 0
330     max2 = 0
331     temp_max = 0
332     temp_orden = None
333     orden1 = None
334     orden2 = None
335     amount1 = {}
336     amount2 = {}
337     for pos in range(0, len(orders)):
338         orden = lt.get_element(orders, pos)
339         if country == orden["Country"] and product == orden["Product"]:
340             count += 1
341             prom_box += float(orden["Price_per_Box"])
342             prom_disc += float(orden["Discount_Pct"])
343             prom_mark += float(orden["Marketing_Spend"])
344             prom_box_ship += int(orden["Boxes_Shipped"])
345             if float(orden["Amount"]) > max1:
346                 max1 = float(orden["Amount"])
347                 max2 = max1
348                 temp_max = max1
349                 temp_orden = orden1
350                 orden1 = orden2
351                 orden2 = temp_orden
352             if count == 0:
353                 return None
354     return None
355
```

```
356     prom_box = prom_box / count
357     prom_disc = prom_disc / count
358     prom_mark = prom_mark / count
359     prom_box_ship = prom_box_ship / count
360
361     if max1 == max2:
362         if float(orden1["Marketing_Spend"]) < float(orden2["Marketing_Spend"]):
363             temp_max = max1
364             max1 = max2
365             max2 = temp_max
366             temp_orden = orden1
367             orden1 = orden2
368             orden2 = temp_orden
369         if max1 == max2:
370             if int(orden1["Order_ID"][4:]) < int(orden2["Order_ID"][4:]):
371                 temp_max = max1
372                 max1 = max2
373                 max2 = temp_max
374                 temp_orden = orden1
375                 orden1 = orden2
376                 orden2 = temp_orden
377
378     amount1 = {
379         "ID" : orden1["Order_ID"],
380         "channel" : orden1["Channel"],
381         "date" : orden1["Order_Date"],
382         "box_ship" : orden1["Boxes_Shipped"],
383         "amount" : max1
384     }
385
```

```
amount2 = {
    "ID" : orden2["Order_ID"],
    "channel" : orden2["Channel"],
    "date" : orden2["Order_Date"],
    "box_ship" : orden2["Boxes_Shipped"],
    "amount" : max2
}

end_time = get_time()
time = delta_time(start_time, end_time)

return time, count, prom_box, prom_disc, prom_mark, prom_box_ship, amount1, amount2
```

Este requerimiento recibe el catálogo de pedidos, un país y un producto, y recorre la totalidad de los pedidos filtrando únicamente aquellos que coinciden con el país y el producto solicitados. Para cada pedido que cumple el filtro, actualiza de forma incremental el conteo y las sumatorias de precio por caja, descuento, gasto en marketing y cajas despachadas, y compara su monto (Amount) contra los dos mayores montos vistos hasta el momento (max1 y max2), conservando también los pedidos correspondientes. Al finalizar el recorrido calcula los promedios, resuelve posibles empates entre los dos montos máximos usando primero el gasto en marketing y luego el Order_ID, y construye los diccionarios con la información de los dos pedidos de mayor monto.

Entrada	catalog, country, product
Salidas	time, count, prom_box, prom_disc, prom_mark, prom_box_ship, amount1, amount2
Implementado (Sí/No)	Si, implementado por todos los integrantes del grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener la lista de pedidos y su tamaño (lt.size)	$O(1)$
Recorrer todos los pedidos con lt.get_element y filtrar por country y product	$O(n)$
Actualizar contadores y sumatorias (precio, descuento, marketing, cajas) del pedido filtrado	$O(1)$ por pedido
Comparar el monto (Amount) y actualizar los dos pedidos de mayor monto (max1, max2)	$O(1)$ por pedido
Calcular los promedios finales (precio, descuento, marketing, cajas)	$O(1)$
Resolver empates entre max1 y max2 (marketing spend y Order_ID)	$O(1)$
Construir los diccionarios de resultado (amount1, amount2)	$O(1)$
TOTAL	$O(n)$

Análisis

El requerimiento tiene complejidad total $O(n)$, siendo n el número de pedidos del catálogo, ya que al no existir una función auxiliar indexada por país y producto es obligatorio recorrer todos los pedidos para filtrar los que cumplen ambas condiciones. Las operaciones dentro del recorrido (sumas, conteo y la comparación para actualizar los dos pedidos de mayor monto) son $O(1)$, por lo que no aumentan el orden del algoritmo. La resolución de empates y la construcción de los resultados finales también son $O(1)$, pues solo involucran los dos pedidos con mayor monto.

La lista de pedidos se implementó como Array List (It), ya que solo se necesita un recorrido secuencial sin inserciones ni eliminaciones, y con el TAD de `get_element()` en $O(1)$ el recorrido completo queda en $O(n)$. Hay que tener en cuenta que con una lista enlazada y acceso por índice el costo habría sido $O(n^2)$. Las pruebas confirman un crecimiento lineal consistente con $O(n)$.